

**M.A.N.T.I.S: A STRATEGY FOR THE EFFECTIVE  
DEVELOPMENT OF A ZERO DEPENDENCY FIRST  
PRINCIPLES HOST BASED INTRUSION DETECTION SYSTEM  
USING MACHINE LEARNING**

By

Anuskar Sigdel

A proposal submitted in partial fulfillment of the requirements for the degree of

BSc (Hons.) Computing

The University of Northampton

## Table of Contents

| SN.                                                     | Page No. |
|---------------------------------------------------------|----------|
| 1. Introduction.....                                    | 1        |
| 1.1. Background.....                                    | 1        |
| 1.2. Rationale .....                                    | 2        |
| 2. Aims and Objectives .....                            | 2        |
| 3. Literature Review .....                              | 3        |
| 4. Research Methodology .....                           | 6        |
| 4.1. Project Activity Overview .....                    | 6        |
| 4.2. Problem Domain Investigation .....                 | 6        |
| 4.3. Software Requirements Engineering .....            | 6        |
| 4.4. System Analysis and Design.....                    | 7        |
| 4.5. System Construction.....                           | 8        |
| 4.6. System Testing.....                                | 8        |
| 4.7. System Evaluation .....                            | 8        |
| 4.8. Algorithm Development .....                        | 8        |
| 5. Professional, Legal and Ethical Considerations ..... | 8        |
| References.....                                         | 9        |
| Appendix.....                                           | 9        |

## List of figures

|                                                                                                          |    |
|----------------------------------------------------------------------------------------------------------|----|
| Figure 1 MANTIS Agile Kanban Methodology Diagram .....                                                   | 7  |
| Figure 2 Gantt Chart .....                                                                               | 10 |
| Figure 3 Comprehensive Architecture and Execution Flow of the MANTIS Intrusion Detection<br>System ..... | 11 |

## **Abbreviations**

API: Application Programming Interface

DDoS: Distributed Denial of Service

GDPR: General Data Protection Regulation

HIDS: Host Based Intrusion Detection System

JSON: JavaScript Object Notation

KNN: K-Nearest Neighbours

MANTIS: Minimalist Alert Network for Threat Intelligence System

OSI: Open Systems Interconnection

PII: Personally Identifiable Information

SIEM: Security Information and Event Management

TCP/UDP: Transmission Control Protocol / User Datagram Protocol

VM: Virtual Machine

XSS: Cross Site Scriptin

## **1. Introduction**

The realm of global cybersecurity operates in an environment defined by relentless escalation. As malicious actors automate the deployment of zero day vulnerabilities and sophisticated ransomware campaigns, targeted organizations must deploy equally sophisticated defense perimeters. The core of this defensive strategy resides in the implementation of Security Information and Event Management (SIEM) systems and Host Based Intrusion Detection Systems (HIDS). These systems act as the primary operational sensors, responsible for ingesting, analyzing, and ultimately blocking anomalous network traffic before critical infrastructure sustains damage.

However, as the velocity of network traffic increases, the foundational engineering behind these defensive systems is being severely tested. Security Operations Centers (SOCs) are demanding faster, intelligently automated responses to novel threats without sacrificing the holistic stability of the underlying operating system. This demand has catalyzed the fusion of low level network engineering with advanced data science, forcing software developers to rethink how network threat telemetry is processed and acted upon.

### **1.1. Background**

Within this specific cybersecurity domain, there is an ongoing architectural shift toward integrating complex machine learning methodologies such as K-Nearest Neighbours and Bayesian probability matrices directly into threat detection telemetry. While integrating advanced clustering theoretically improves mitigation rates, the practical implementation has inadvertently birthed abstract dependency bloat.

Currently, the vast majority of commercial and open source HIDS platforms are enormous, computationally demanding ecosystems such as Zeek or Wazuh. They rely intensely on massive external machine learning packages, such as Scikit-Learn or TensorFlow. This reliance on imported, black box third party resources create an environment where foundational mathematics governing threat identification are entirely opaque. On edge network servers, these massive libraries consume an unacceptable amount of system memory simply to load their dependencies, making them inefficient for rapid deployment.

## 1.2. Rationale

The primary problem this proposed project addresses is the industry over reliance on bloated, imported machine learning libraries and external frameworks in modern cybersecurity tools. Relying on massive dependencies hides the algorithmic decision making process, places a severe computational burden on the host machine, and drastically widens the attack surface for software supply chain vulnerabilities historically evidenced by catastrophic, dependency based breaches like the SolarWinds incident.

Solving this is significant because investigating a mathematically transparent HIDS utilizing strictly the core Python Standard Library improves network defense efficiency. Returning to architectural first principles results in clear benefits: the complete eradication of third party supply chain risks, total algorithmic transparency for human analysts, and a significantly smaller CPU footprint. The motivation for building MANTIS is to prove advanced machine learning can execute with zero external dependencies and still compete with commercial bloated software.

## 2. Aims and Objectives

With the fundamental objective of the proposed research being the design, development, and evaluation of the M.A.N.T.I.S zero dependency Intrusion Detection framework, there are a number of specific research goals that must be carefully outlined and achieved over the course of the dissertation.

- A detailed evaluation of the mathematical baselines such as Standard Deviation for volume flow, Euclidean Distance for behavioral clustering, and Bayesian Probability for text inspection necessary for machine learning threat classification without reliance on external libraries.
- The development of a secure, unbuffered raw socket packet capture mechanism at OSI Layer 2 that interacts directly with the Linux Kernel for zero overhead telemetry ingestion, prioritizing speed and system stability over high level abstraction.

- The construction of independent analytical detection engines from scratch that mathematically model volumetric floods, behavioral port scans, and malicious payload injections perfectly in native Python code.
- The integration of an autonomous, OS native active defense strategy that dynamically leverages local iptables configurations to block identified threats immediately without requiring a human operator to intervene in real time.
- The design of an asynchronous, multi threaded communication pipeline that efficiently connects the core sensor node to a lightweight HTTP telemetry server and web dashboard so that alerts can be viewed safely on another device in the network.
- The formulation of a live fire penetration testing strategy using standard offensive tools, such as Nmap and hping3, to rigorously evaluate the system performance and accuracy inside an isolated virtual lab environment.

### 3. Literature Review

The intersection of low level systems programming and high level data science has been heavily documented in recent cybersecurity literature. A complete understanding of the theoretical work completed by previous researchers is required to properly navigate the challenges of building a reliable network defense mechanism from scratch. The following peer-reviewed papers establish the foundational baseline for this project's mathematical approach.

#### **Machine Learning-Based Intrusion Detection System: An Experimental Comparison**

**Author:** Hidayat, I., Ali, M. Z., and Arshad, A.

**Date:** 2022

This empirical 2022 study conducted a comprehensive examination of contemporary machine learning paradigms against modern network intrusions. The authors established a significant conclusion, which is that isolated, specialized ML mathematical algorithms when tuned distinctly for strict network environments can vastly outperform generalized, heavyweight framework ensembles in pure edge-computing ecosystems. They particularly highlighted that algorithmic

models tightly coupled to the data processing layer reduce CPU inference latency significantly compared to heavily abstracted ML libraries.

This research directly validates the core philosophy of MANTIS's Zero-Dependency architecture. By proving that concisely tuned, standalone ML equations can outpace abstracted libraries, Hidayat and the subsequent authors strongly support MANTIS's methodology of constructing Z-Score volume trackers purely from scratch in Python. Their findings mathematically prove that it is entirely possible to completely eradicate CPU bloat without sacrificing high detection fidelity, especially when the algorithm is optimized for the exact packet structures it receives.

### **A False Sense of Security? Revisiting the State of Machine Learning-Based Industrial Intrusion Detection**

**Author:** Kus, D., Wagner, E., Pennekamp, J., Wolsing, K., Fink, I. B., Dahlmanns, M., Wehrle, K., and Henze, M.

**Date:** 2022

Published in the 2022 ACM CPSS Workshop, this foundational paper explored the critical blind spots of modern ML Intrusion Detection systems. The authors proved mathematically that models reliant on massive, pre-calculated generalized training datasets consistently fail to recognize novel, zero day anomalous attacks when operating inside live, continuous runtime environments. The primary reason they fail is that fixed training sets cannot understand dynamic network topography or subtle, gradual shifts in legitimate user behavior over time.

This powerfully justifies the MANTIS Behavioral clustering approach. Because static training sets offer a false sense of security, MANTIS mandates dynamic, real time K-Nearest Neighbours distance heuristics. Re-creating these distance formulas mathematically allows the sensor to profile anomalous clustering directly from immediate local traffic. This bypasses the extreme flaws of static AI models by ensuring the detection engine is constantly adapting to the actual environment rather than relying on outdated, generalized algorithmic assumptions.

## **Applying Generative Machine Learning to Intrusion Detection: A Systematic Mapping Study and Review**

**Author:** Halvorsen, J., Izurieta, C., Cai, H., and Gebremedhin, A.

**Date:** 2024

This 2024 study mapped the absolute leading edge of probabilistic and generative machine learning models. The authors discovered that while advanced generative probability arrays are highly accurate at diagnosing complex text payloads, their immense software latency severely cripples their ability to execute inline TCP blocking on resource constrained systems. Heavy machine learning essentially processes data incredibly accurately but far too slowly to actively block a high speed network attack during a live runtime scenario.

This 2024 ACM research perfectly supports the MANTIS Payload Engine architecture. It confirms that probabilistic calculations such as Naive Bayes arrays are ideal for parsing malicious SQL injection payloads, but they must be implemented with near zero software latency. MANTIS satisfies this constraint by computing these text probability matrices mathematically by hand via the Python Standard Library. This avoids the severe processing overhead involved in calling external inference engines, theoretically enabling immediate packet dropping.

## **Network Intrusion Detection System Using XGBoost and Random Forest Algorithms**

**Author:** Onyebueke, A. E., David, A. A., and Munu, S.

**Date:** 2023

This recent 2023 study evaluated modern Machine Learning models utilizing XGBoost and Random Forest algorithms to anticipate and successfully identify dynamic network intrusions,

establishing new benchmarks for contemporary classification speeds against novel vectors. The researchers demonstrated that decision-tree style ensembles are particularly adept at identifying the subtle behavioral fingerprints left by sophisticated port scanning and reconnaissance tools before a full attack is launched.

This modern research validates the MANTIS behavioral engine design constraints. Since ensemble algorithms have proven highly effective at predicting modern intrusions through behavioral profiling, MANTIS leverages this mathematical framework to justify utilizing distance based clustering heuristics built entirely from scratch in native Python. By calculating the Euclidean distance between connection attempts, MANTIS captures these exactly identified behavioral patterns without inheriting the massive CPU overhead associated with importing Random Forest libraries.

## **4. Research Methodology**

### **4.1. Project Activity Overview**

The primary methodology applied is the Constructive Research Methodology, validating theoretical models through the construction and benchmarking of an IT artifact. Working in tandem, the development process will be driven by the Agile Kanban SDLC. Because mathematical thresholds require iterative observation and immediate feedback loops against network traffic, a rigid Waterfall approach is fundamentally unsuited.

### **4.2. Problem Domain Investigation**

Investigation focuses on Linux raw socket programming and manual parsing of raw Ethernet, IP, TCP, and UDP headers. Evaluating mathematical performance constraints of native Python versus compiled languages will be explored. Existing massive black box SIEMs will be contrasted with this lightweight approach.

### **4.3. Software Requirements Engineering**

Requirements gathering will concentrate on defining exact packet structures, extracting 14 byte Ethernet frame identifiers, and developing logical formulations required to parse live traffic arrays

into usable analytical variables, ensuring the sensor operates concurrently. System limitations and capabilities must be strictly defined to prevent feature creep.

The core cyclical framework of the MANTIS Agile Kanban methodology is illustrated in Figure

1: -

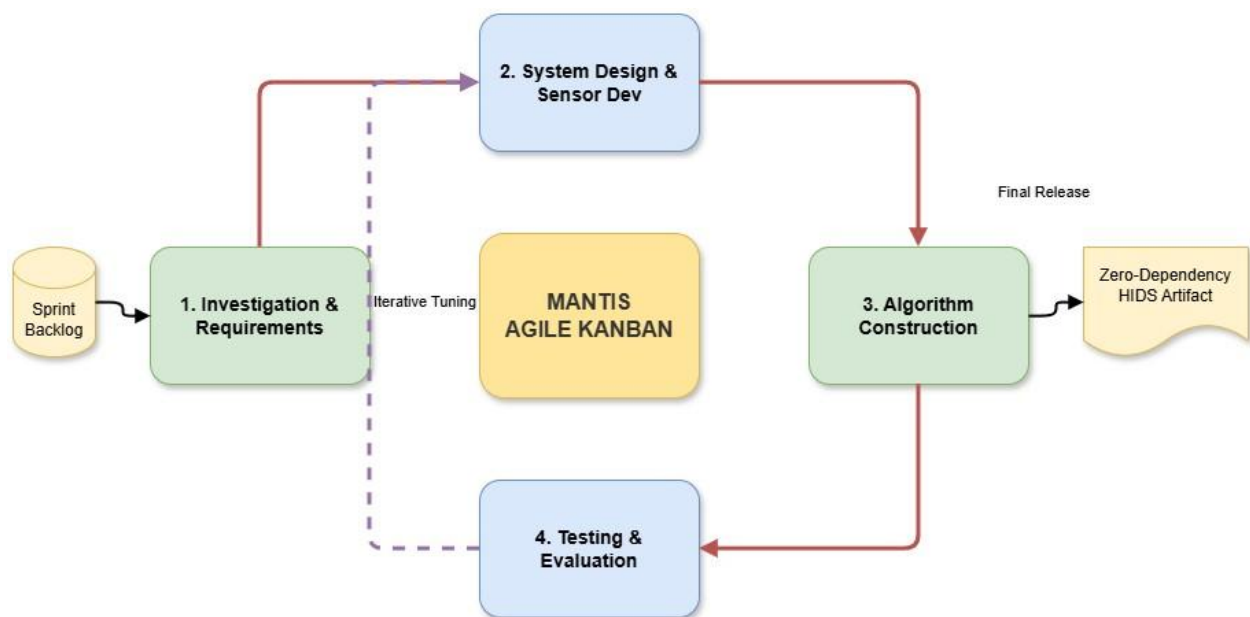


Figure 1 MANTIS Agile Kanban Methodology Diagram

As depicted in Figure 1, development is driven by a continuous Kanban backlog. Rather than a linear approach, the system progresses through iterative loops bridging System Design, Algorithm Construction, and Evaluation. This cyclical 'Iterative Tuning' ensures constant security refinement before outputting the final zero-dependency artifact

#### 4.4. System Analysis and Design

The architectural design will employ a threaded, decoupled flow model. The sensor will act independently from the SIEM UI dashboard, bridging communication asynchronously via JSON structures over UDP ports. This ensures web UI computational rendering delays do not bottleneck real time packet ingestion speeds.

#### **4.5. System Construction**

To enforce independence, programming relies exclusively on standard Python 3 libraries in a Kali Linux OS environment, chosen to allow native firewall manipulation without third party systems. The dashboard UI strictly utilizes HTML and vanilla JS over an HTTP socket server. Code is structured modularly to allow independent algorithm analysis.

#### **4.6. System Testing**

The testing strategy involves a modified V-Model approach aligned with the Agile framework. Algorithm Test cases involve mathematical validity unit tests utilizing static Python lists, proving formulas calculate accurately before observing real data. Following validation, full integration testing will verify the pipeline flow from physical network interfaces to the browser dashboard.

#### **4.7. System Evaluation**

Evaluation will be rigorously quantitative. The project will leverage structured live fire testing using offensive network tools against the operational sensor within an isolated Virtual Machine testing enclave, supplemented by the CIC-IoT2023 dataset to benchmark accuracy against an academic standard. System success will be empirically evaluated by computing True Positive alerts versus False Positives under active attack loads.

#### **4.8. Algorithm Development**

Algorithms will be explicitly constructed from first principles. This involves writing customized standard deviation logic for volumetric attacks, Euclidean distance KNN models for behavioral profiling, and Bayesian calculations for text payload abnormalities precisely by hand.

### **5. Professional, Legal and Ethical Considerations**

Creating security tools mandates ethical boundary setting. No experimental volunteers are used. There is a duty to ensure the publication of algorithms does not provide malicious actors evasion models. The research maintains impartiality by executing standard, unmodified offensive payloads utilizing standard Nmap flags without tuning attacks to favor the engine.

Legally, the project complies with the UK Computer Misuse Act (1990). The proposal guarantees no active cyber attacks or port scanning sequences will ever traverse public internet infrastructure. All tooling simulations will be isolated entirely within private Virtual Machine boundaries completely owned by the researcher on local hardware.

Furthermore, in accordance with the Data Protection Act (2018) and GDPR standards, the packet sniffer is configured not to capture or store real world Personally Identifiable Information (PII).

The core sensor will solely parse locally generated network metadata and synthetic testing payloads, guaranteeing absolute privacy compliance during testing phases.

## **References**

Python Software Foundation (2024) Python 3.12 Documentation. Available at: <https://docs.python.org/3/> (Accessed: 15 January 2026).

National Institute of Standards and Technology (NIST) (2024) Intrusion Detection Systems (IDS). Available at: [https://csrc.nist.gov/glossary/term/intrusion\\_detection\\_system](https://csrc.nist.gov/glossary/term/intrusion_detection_system) (Accessed: 19 February 2026).

Agile Alliance (2024) What is Agile? Available at: <https://www.agilealliance.org/agile101/> (Accessed: 10 February 2026).

Linux Kernel Organization (2024) The Linux Kernel Documentation. Available at: <https://kernel.org/doc/> (Accessed: 26 February 2026).

## **Appendix**

### **Appendix 1: Project Plan Gantt Chart**

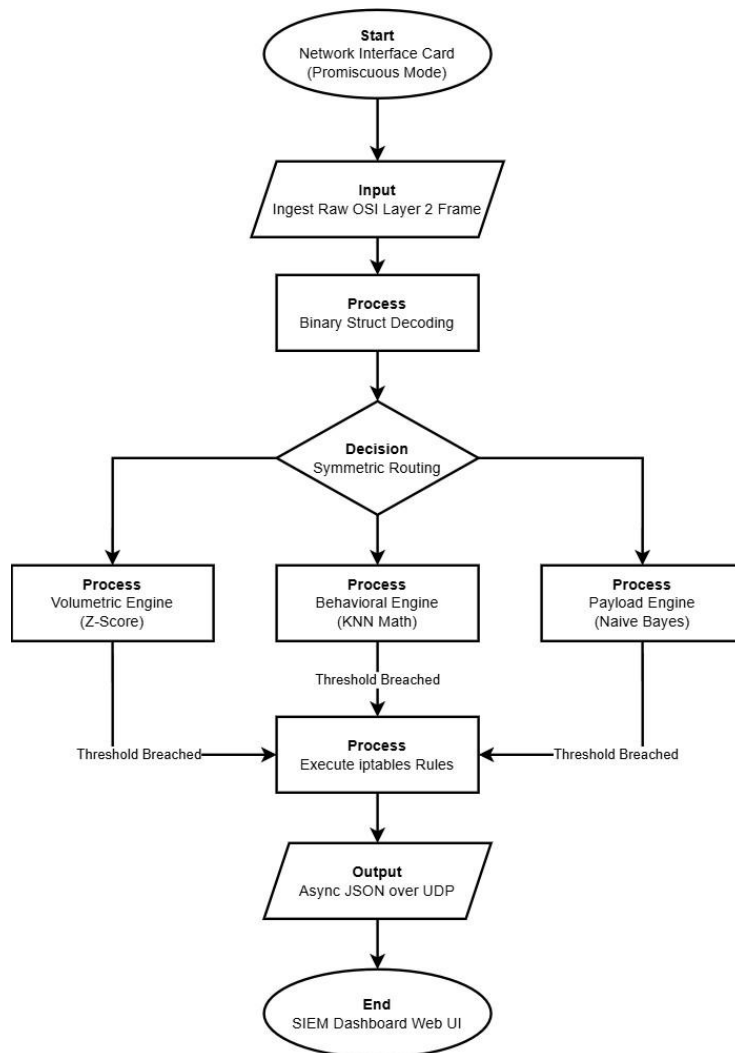
The project developmental timeline, broken into five iterative Agile Sprints, is shown in Figure 2:

-



## Appendix 2: Proposed System Flow Diagram

The step-by-step process of how MANTIS actually catches network attacks is shown below in Figure 3: -



**Figure 3 Comprehensive Architecture and Execution Flow of the MANTIS Intrusion Detection System**

As shown in Figure 3, the system continuously ingests and decodes raw network traffic. Data routes simultaneously to three independent detection engines (Volumetric, Behavioral, and Payload). If any engine's mathematical threshold is breached, MANTIS automatically

executes *iptables* blocking rules and instantly transmits JSON alerts to the administrative web dashboard.